



Agentic Intelligence for Automated Document Extraction and Analysis

BY : Rebel_AI

Index :



01	•	Project Overview
02	•	Technology Stack
03	•	Manager agent & Work Flow
04	•	User client and Work flow
05	•	Internal Communication
06	•	Sub-Agents and work Flows
07	•	API Architecture & Postman Interaction
08	•	System Logic & Cloud Integration
09	•	Project Impact & Future Enhancement
10	•	Conclusion

Project Overview :

01

The project presents a multi-agent architecture built using Agentic AI principles, aimed at intelligent processing of PDF documents. It includes a centralized manager agent and three specialized sub-agents working collaboratively.

The parameters include :

- The system is composed of a **Manager Agent** and three specialized **Sub-agents** :
- **Librarian:** Extracts structured content from PDFs.
- **Artist:** Analyzes visual styling (fonts, colors.
- **Bookkeeper:** Loads CSV files into SQLite database and creates requested graphs.



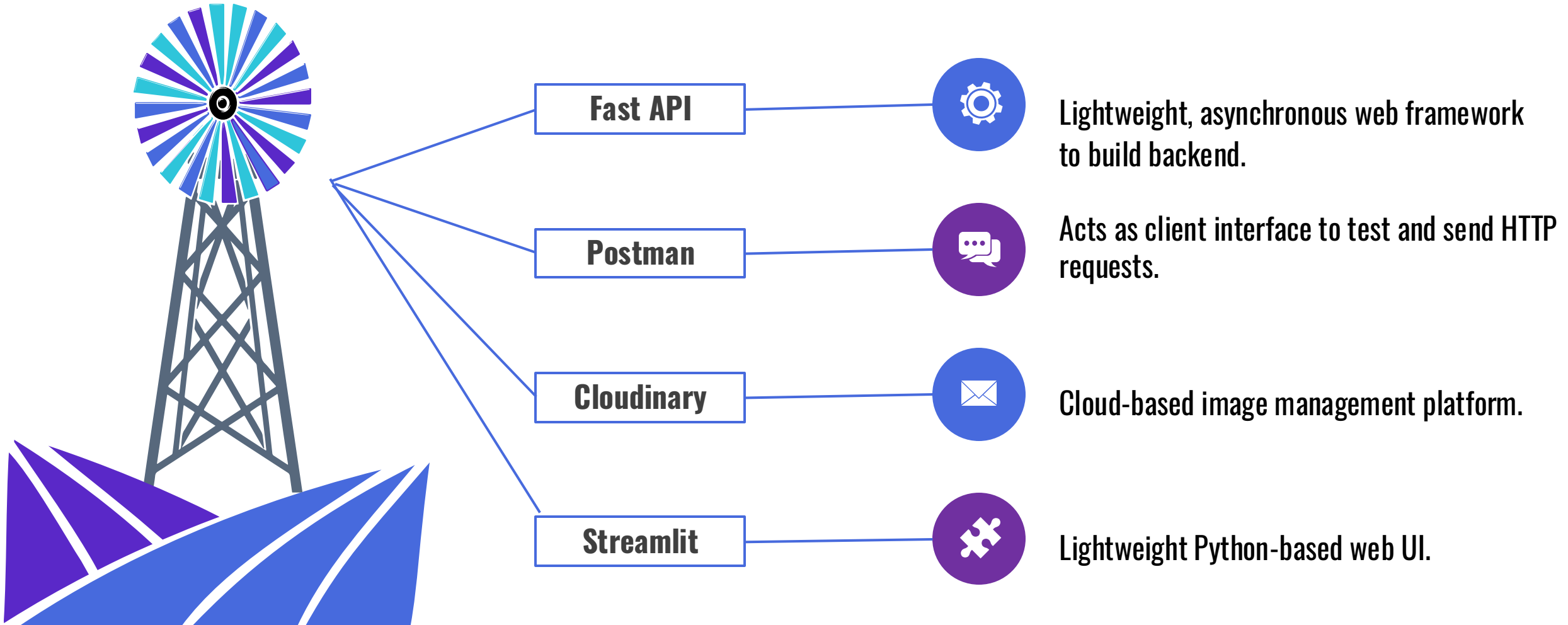


Key capabilities :

- The system uses **agent-based reasoning** to divide and manage complex document **automating PDF analysis**, transforming unstructured documents into actionable data.
- Manager Agent acts as the **central coordinator**, handling user queries and delegating tasks to sub-agents.flow.

Technology Stack :

03

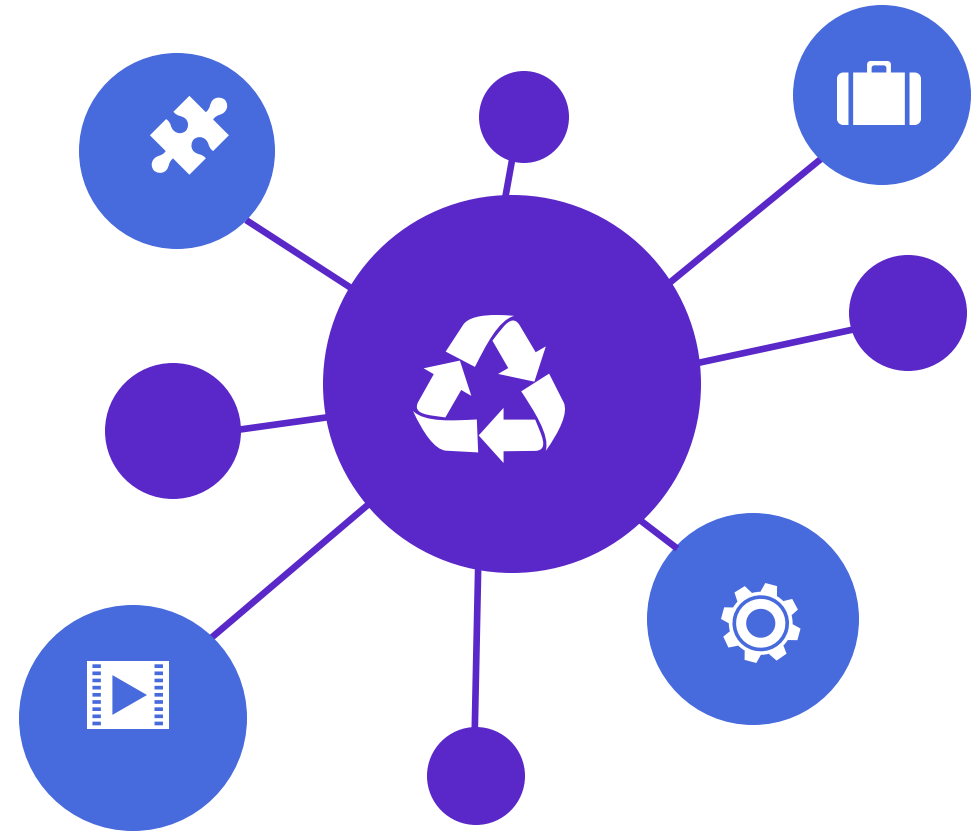


Handles User Input :

- Receives uploaded PDFs and chat-based queries via FastAPI endpoints.
- Acts as the central interface between the user and the agent system.

Decides Query Routing :

- Analyzes the user's query to determine which sub-agent (Librarian, Artist, or Bookkeeper) is most appropriate for handling it.
- Uses rule-based logic or keyword mapping to decide routing.



Inter-Agent Communication :

- Sends the query and any necessary data to the selected sub-agent.
- Waits for the response and logs the result.

Validates Agent Responses :

- The Manager Agent validates if the sub-agent's response matches the query intent.
- If relevant, it returns it to the user, otherwise, it reroutes the query to another sub-agent.



Manager Agent – Workflow :

06



Input Stage:

Accepts:

- POST /upload_pdf: Stores the PDF.
- POST /chat: Processes a query about the uploaded PDF.



Decision Stage:

- Identifies the nature of the task: extraction, visual analysis, or data formatting.



Delegation Stage:

- Sends the request to the appropriate sub-agent.
- Supplies relevant file paths, table structures.



Evaluation Stage:

- Validates sub-agent's response.
- If necessary, retries with a different sub-agent.



Output Stage:

- Sends response to user via API.
- Asks user: "Do you need further assistance?"

User Interaction – Workflow :

07

Step 1: User uploads a PDF using the POST /upload_pdf endpoint in Postman.



Step 2: System stores the PDF temporarily for processing.



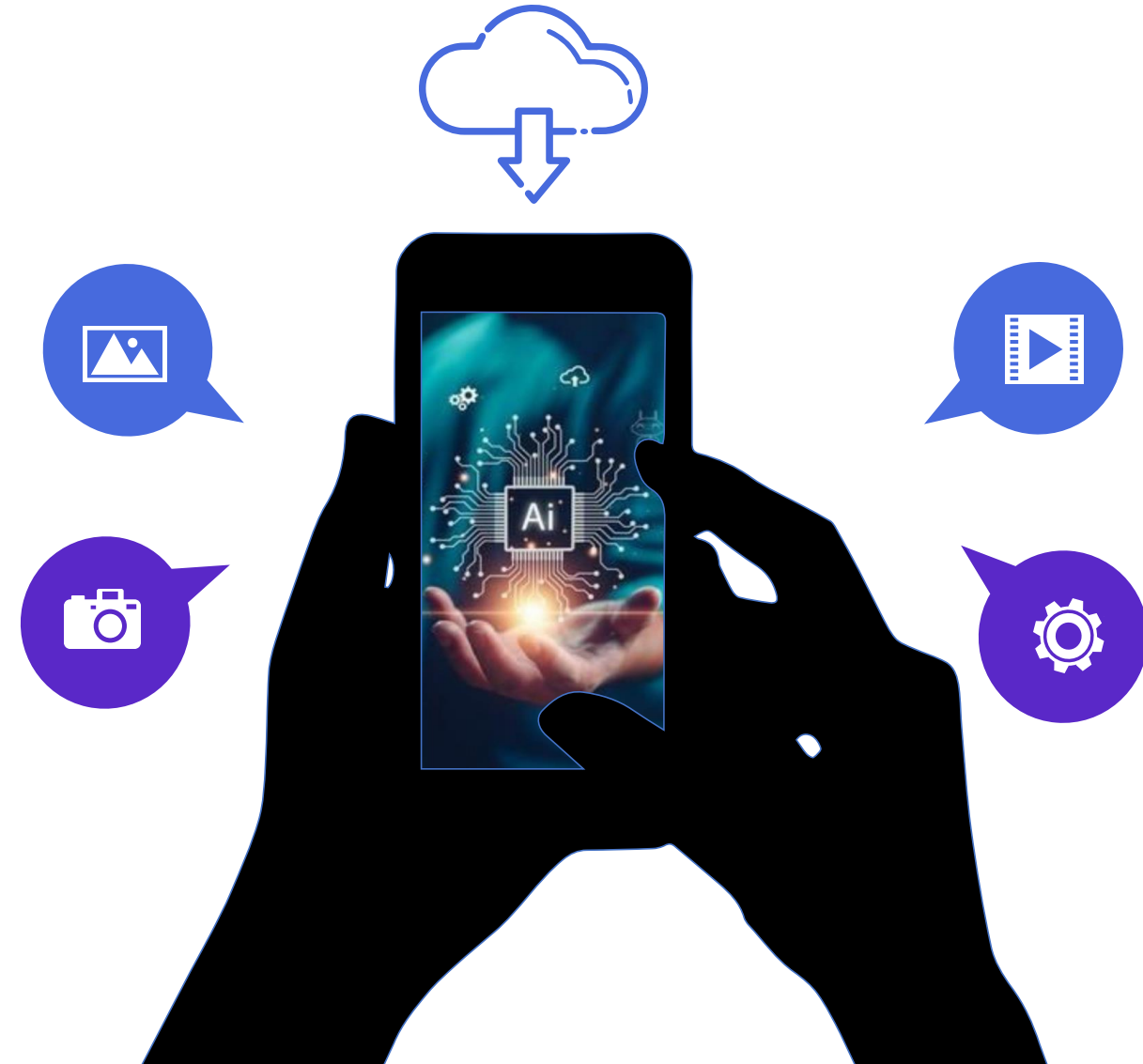
Step 3: User sends a query through the POST /chat endpoint.



Step 4: Manager Agent receives the query, initiates processing, and returns the response.

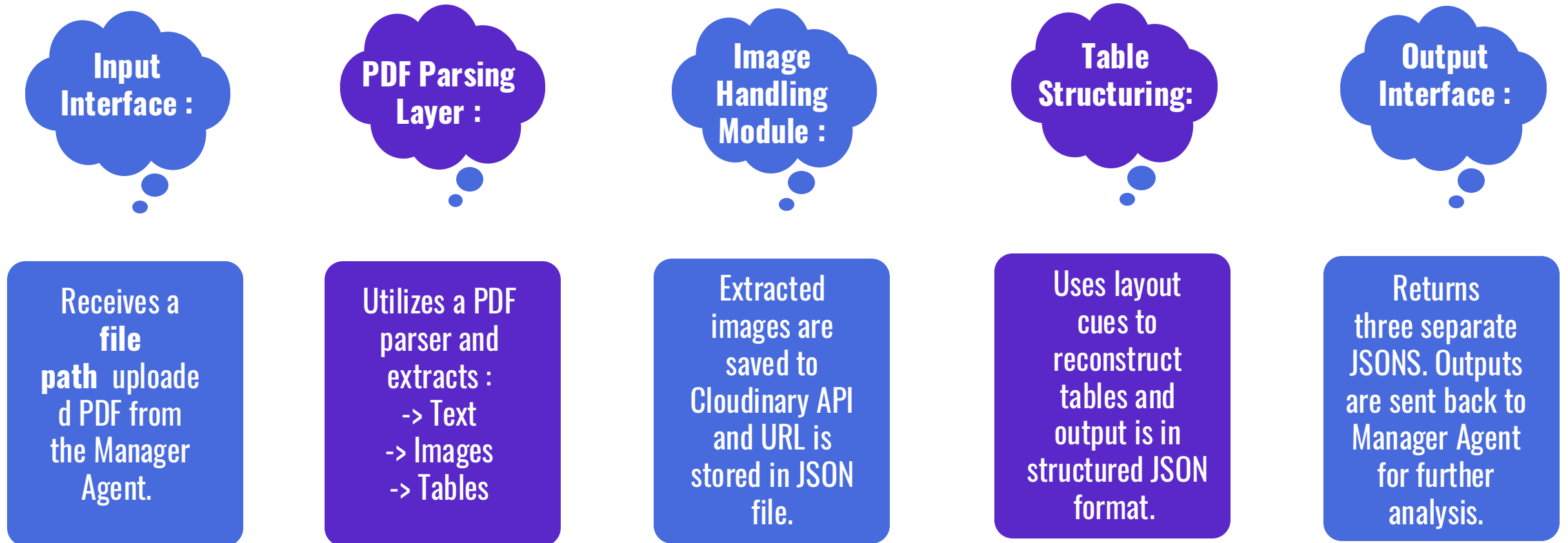
Introduction to Sub-Agent 1 :

- The **Librarian Agent** is responsible for extracting structured information from uploaded PDF documents.
- It acts as the system's **data acquisition component**, parsing the content into three main types—text, images, and tables.
- The extracted data is formatted into JSON, enabling smooth communication.
- This agent ensures that raw document content is efficiently converted into usable digital data.



Librarian Agent – Architecture :

09



Introduction to Sub-Agent 2 :

Responsible for **visual content analysis** of images extracted from user-submitted PDFs..



Accessible via a FastAPI POST endpoint (/analyze-image) that accepts image files and optional text queries.

Performs tasks such as:

- Color palette analysis
- Font style and family detection
- Logo detection and metadata extraction



Uses **GPT-4o** for interpreting visual features and generating structured outputs.

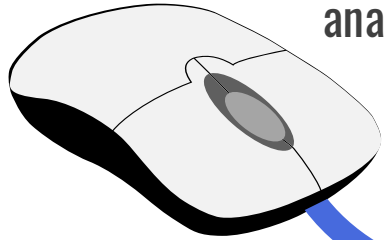


Artist Agent - Architecture :

11

Input Stage :

Accepts individual image files and Optionalizes user query for focused analysis



Output & Logging :

- Logs results including color, font, and logo information.
- Structured data is returned via FastAPI.



Final Output :

Sends analysis back to the Manager Agent.



Processing Layer :

FastAPI interface receives and processes requests and Analyzes:

- Dominant color hex codes
- Font families and styles
- Logos and associated metadata

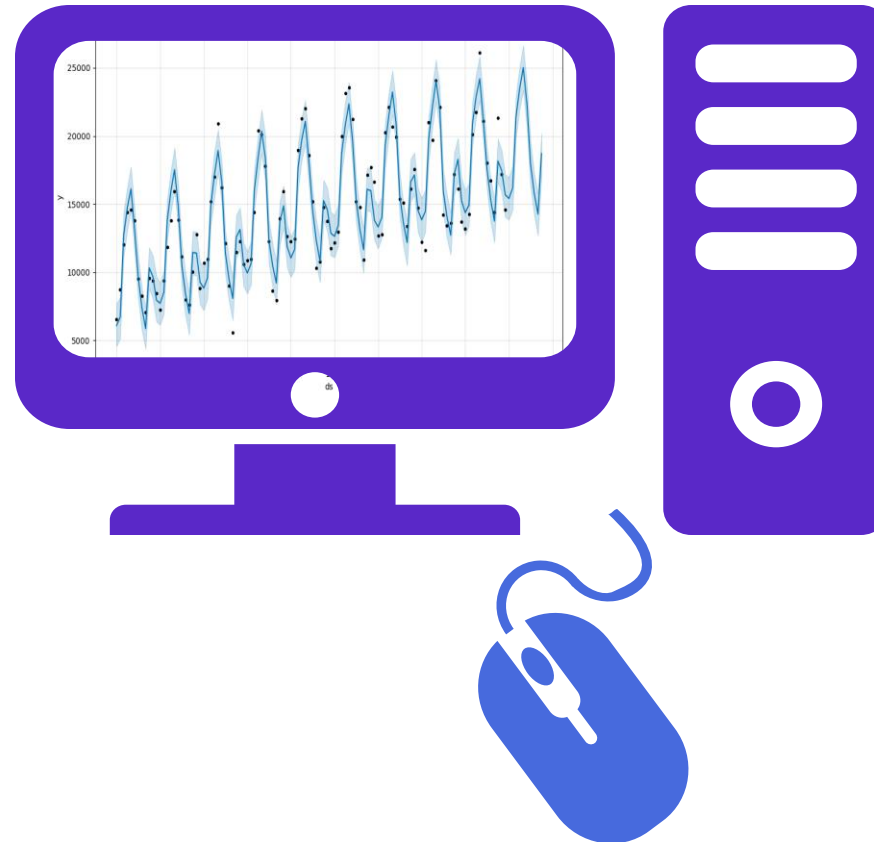
Data Management :

- Stores image data and analysis results in a vector database.
- Uses FAISS indexing.

Introduction to Sub-Agent 3 :

Purpose:

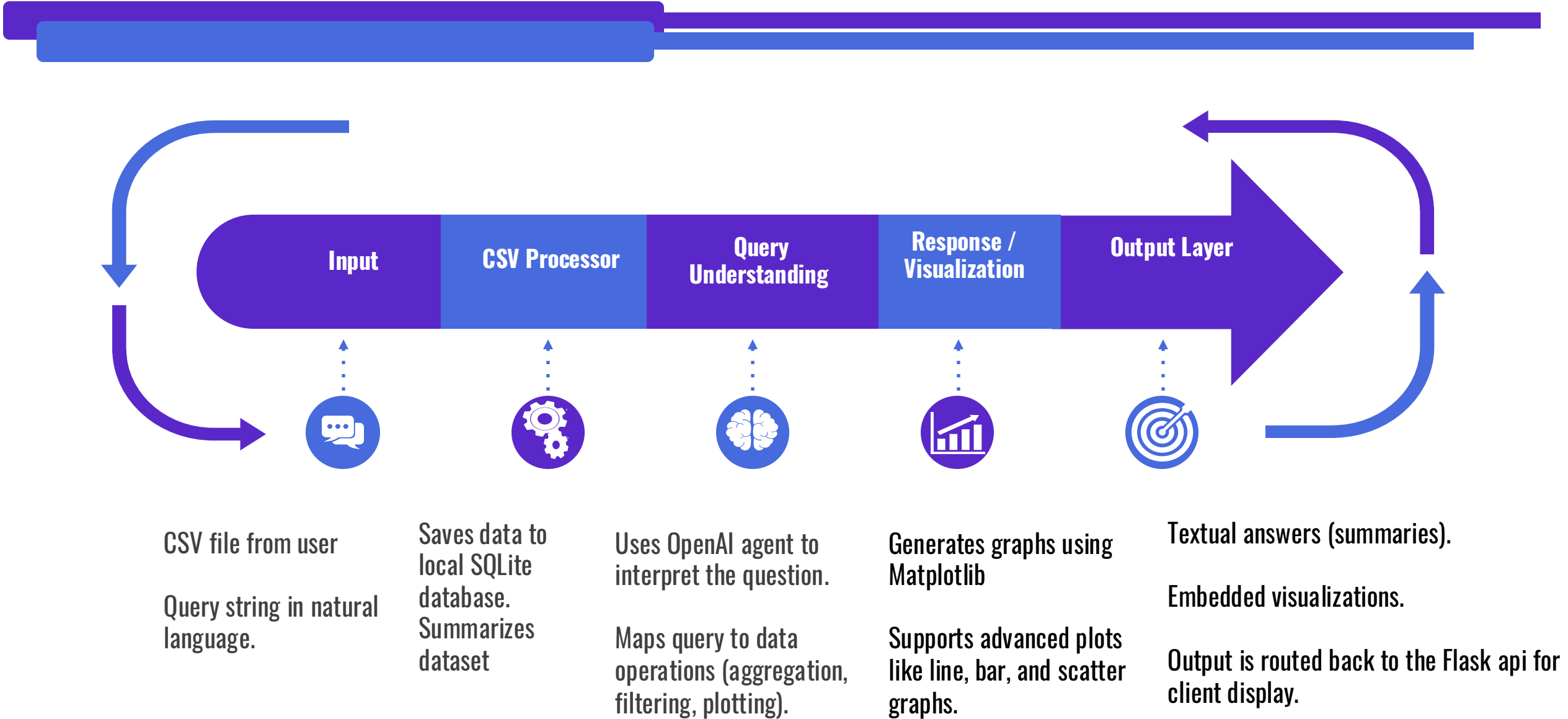
- Acts as an Intelligent CSV Analyzer Agent.
- Handles uploaded CSV files for data inspection, Q&A, and visualization.



Key Responsibilities:

- Uploads data to local database.
- Answers natural language queries such as:
 - “What is the average temperature?”
 - “List average snowfall by year.”
- Generates smart visualizations:
 - Plots time-series graph.
 - Line charts
 - Bar plots

Book-Keeper - Architecture :



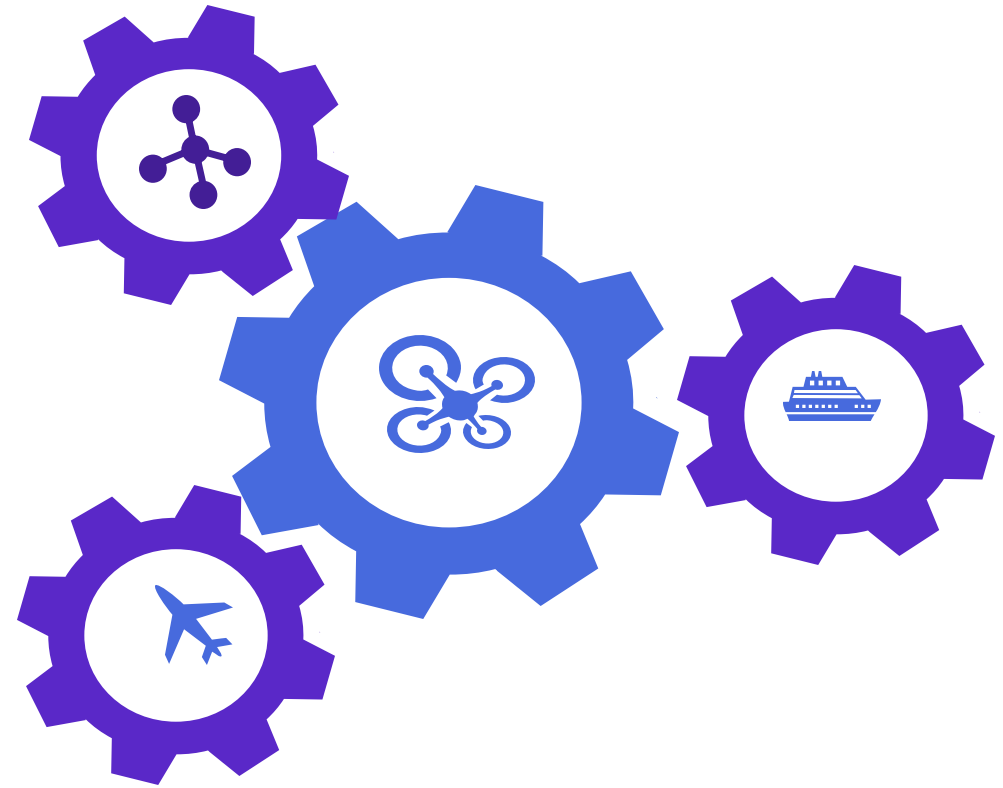
API Architecture :

FastAPI Endpoints:

- POST /upload_pdf – Upload a PDF.
- POST /chat – Submit queries.

API Interaction :

- Upload PDF → Manager distributes to agents.
- Chat with system → Manager routes to agents and compiles responses.





System Response Logic :

Manager Agent's Role in Validation:

- Evaluates if the sub-agent's answer matches the query intent.
- **If relevant:** response is sent to the user.
- **If irrelevant:** query is redirected to another agent.

Goal:

- Ensure accurate and contextually relevant output every time.

Result:

- Only the best-fit answer is returned to the user after internal verification.



Project Impact :

- Automates tedious document analysis tasks.

Can be applied in:

- Legal: Summarize case documents.
- Medical: Parse patient reports or scans.
- Academia: Process research PDFs and extract visuals/tables.
- Reduces human error and increases throughput in data handling.



Future Enhancements :

- Expand file format support to DOCX, XLSX, scanned images (OCR).
- Incorporate Natural Language Understanding (NLU) to interpret queries more contextually.
- Develop a multi-user collaborative interface (e.g., web dashboard).
- Introduce learning-based routing (use ML to predict which agent should handle the query).

Conclusion :

-->Successfully implemented a multi-agent system using Agentic AI principles.

-->The system handles PDF processing, content extraction, visual analysis, and data visualization through coordinated sub-agents.

-->Manager Agent plays a central role in routing user queries and validating responses from sub-agents.

-->Achieved smooth integration with tools like FastAPI, Postman, and Cloudinary.



Q & A :

Q1: Why did you choose FastAPI over other web frameworks?

Q2 : How does the Manager Agent decide which sub-agent should handle a user query?

Q3: How is Cloudinary integrated, and what is its role in the system?

Q4: How does the system ensure data privacy and security?

Q5: What future improvements are planned for intelligent query routing?

Q6: Can the system be deployed in a private cloud or local environment?



THANK YOU

REBEL_AI